Nama: Muhammad Fadhel

Nim: 123140106

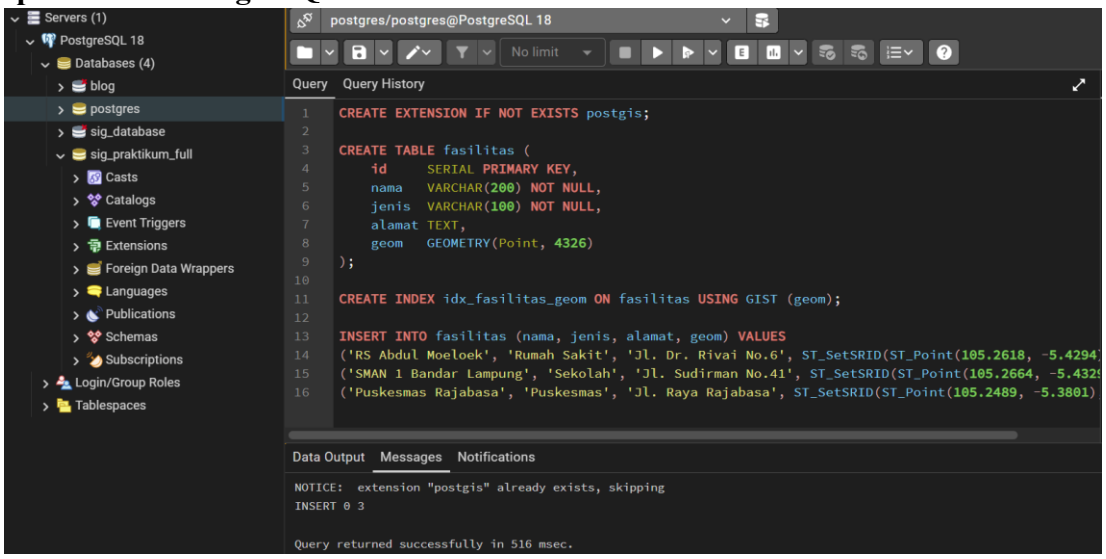
Tugas: P7

1. Persiapan Project

```
PS C:\webgis-api> cd..
PS C:\> mkdir webgis-api
>> cd webgis-api
>> python -m venv venv
>> venv\Scripts\activate
>> pip install fastapi uvicorn asyncpg python-dotenv
>> pip freeze > requirements.txt^C
PS C:\> & c:\webgis-api\.venv\Scripts\Activate.ps1
(.venv) PS C:\>
```

Membuat virtual environment Python dan menginstall seluruh dependencies yang diperlukan yaitu FastAPI, Uvicorn, asyncpg, dan python-dotenv.

2. Setup Database PostgreSQL



Membuat tabel fasilitas pada database PostgreSQL dengan ekstensi PostGIS aktif. Tabel memiliki kolom id, nama, jenis, alamat, dan kolom geometri (geom) bertipe Point dengan SRID 4326 (WGS84). Spatial index GIST dibuat agar query spasial berjalan lebih cepat.

3. Buat File .env

```
.env
DATABASE_URL=postgresql://postgres:pw@localhost:5432/postgres
```

Membuat File .env menyimpan konfigurasi koneksi database agar tidak hardcoded di dalam kode. File ini dibuat di folder utama project

4. Buat database.py

```python
import asyncpg
from dotenv import load_dotenv
import os

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
pool = None

async def get_pool():
    global pool
    if pool is None:
        pool = await asyncpg.create_pool(DATABASE_URL, min_size=5, max_size=20)
    return pool

async def close_pool():
    global pool
    if pool:
        await pool.close()
        pool = None
```

File database.py mengkonfigurasi connection pool menggunakan asyncpg. Pool dibuat dengan minimum 5 dan maksimum 20 koneksi, sehingga request dapat dilayani secara efisien tanpa membuat koneksi baru setiap saat.

5. Buat models.py

```python
from pydantic import BaseModel, Field
from typing import Optional

class FasilitasCreate(BaseModel):
    nama: str = Field(..., min_length=3)
    jenis: str
    alamat: Optional[str] = None
    longitude: float = Field(..., ge=-180, le=180)
    latitude: float = Field(..., ge=-90, le=90)
```

File models.py mendefinisikan schema Pydantic FasilitasCreate untuk validasi input. Pydantic memastikan bahwa nilai longitude berada dalam rentang -180 hingga 180, latitude -90 hingga 90, dan nama minimal 3 karakter.

6. Buat routers/fasilitas.py

routers > fasilitas.py > delete

```python
import json
from fastapi import APIRouter, HTTPException, Query
from Database import get_pool
from models import FasilitasCreate

router = APIRouter(prefix="/api/fasilitas", tags=["Fasilitas"])

# GET ALL
@router.get("/")
async def get_all():
    pool = await get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("""
            SELECT id, nama, jenis, alamat,
                   ST_X(geom) AS longitude, ST_Y(geom) AS latitude
            FROM fasilitas ORDER BY id
        """)
    return [dict(r) for r in rows]
```

File ini berisi seluruh endpoint API mulai dari GET all, GET by ID, GET GeoJSON, GET Nearby (query spasial), POST, PUT, hingga DELETE. Semua query menggunakan parameterized query ($1, $2, ...) untuk mencegah SQL injection.

7. Buat main.py

main.py > root

```python
from fastapi import FastAPI
from contextlib import asynccontextmanager
from Database import get_pool, close_pool
from routers import fasilitas

@asynccontextmanager
async def lifespan(app: FastAPI):
    await get_pool()
    print("Database connected")
    yield
    await close_pool()

app = FastAPI(
    title="WebGIS API",
    description="API data spasial PostGIS",
    version="1.0.0",
    lifespan=lifespan
)

app.include_router(fasilitas.router)

@app.get("/")
async def root():
    return {"message": "WebGIS API aktif", "docs": "/docs"}
```

File main.py adalah entry point aplikasi. Di sini FastAPI diinisialisasi dengan lifespan context manager untuk mengelola startup dan shutdown connection pool. Router fasilitas juga didaftarkan di sini.

8. Jalankan Server

```
(venv) PS C:\webgis-api> & "c:\webgis-api\venv\Scripts\uvicorn.exe" main:app --re
load
INFO:     Will watch for changes in these directories: ['C:\\webgis-api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [19360] using StatReload
INFO:     Started server process [27240]
INFO:     Waiting for application startup.
```

Server dijalankan menggunakan perintah uvicorn main:app --reload dari dalam folder project dengan virtual environment aktif. Flag --reload memungkinkan server restart otomatis saat ada perubahan kode

9. Test via Swagger UI

Semua endpoint diuji melalui Swagger UI yang tersedia otomatis di http://localhost:8000/docs. Berikut hasil pengujian setiap endpoint:

Method	Endpoint	Fungsi	Status
GET	/api/fasilitas	Ambil semua data	200 OK
POST	/api/fasilitas	Tambah data baru	201 Created
GET	/api/fasilitas /geojson	Output GeoJSON	200 OK
GET	/api/fasilitas?/ nearby	Cari fasilitas terdekat	200 OK
GET	/api/fasilitas /{id}	Ambil by ID	200 OK
PUT	/api/fasilitas /{id}	Update data	200 OK
DELETE	/api/fasilitas /{id}	Hapus data	204 No Content

- **/api/fasilitas/Get All**

200

Response body


```
[
  {
    "id": 2,
    "nama": "SMAN 1 Bandar Lampung",
    "jenis": "Sekolah",
    "alamat": "Jl. Sudirman No.41",
    "longitude": 105.2664,
    "latitude": -5.4329
  },
  {
    "id": 3,
    "nama": "Puskesmas Rajabasa",
    "jenis": "Puskesmas",
    "alamat": "Jl. Raya Rajabasa",
    "longitude": 105.2489,
    "latitude": -5.3801
  },
  {
    "id": 4,
    "nama": "string",
    "jenis": "string",
    "alamat": "string",
    "longitude": -180,
    "latitude": -90
  },
  {
    "id": 5,
    "nama": "string",
```


Download

Endpoint ini mengembalikan seluruh data fasilitas yang tersimpan di database. Menggunakan fungsi PostGIS ST_X(geom) dan ST_Y(geom) untuk mengekstrak koordinat longitude dan latitude dari kolom geometri.

- **/api/fasilitas/create**

Code | Details

201

Response body


```
{
  "id": 7,
  "nama": "Institut Teknologi Sumatera",
  "jenis": "Universitas",
  "alamat": "Jl. Terusan Ryacudu, Way Huwi, Lampung Selatan",
  "longitude": 105.3107,
  "latitude": -5.3621
}
```


Download

Response headers

```
content-length: 165
content-type: application/json
date: Sat,04 Apr 2026 16:01:30 GMT
server: uvicorn
```

Endpoint POST digunakan untuk menambahkan fasilitas baru. Input divalidasi otomatis oleh Pydantic sebelum disimpan ke database. Koordinat dikonversi ke geometri PostGIS menggunakan ST_SetSRID(ST_Point(lon, lat), 4326). Mengembalikan data yang baru dibuat beserta ID auto-generated (status 201 Created)

- **/api/fasilitas/Get Geojson**

Code | Details

200

Response body


```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          105.2664,
          -5.4329
        ]
      },
      "properties": {
        "id": 2,
        "nama": "SMAN 1 Bandar Lampung",
        "jenis": "Sekolah"
      }
    },
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          105.2489,
          -5.3801
        ]
      },
      "properties": {
```


Download

Endpoint ini mengembalikan data dalam format GeoJSON FeatureCollection yang merupakan standar internasional untuk data geografis. Menggunakan fungsi PostGIS ST_AsGeoJSON(geom) untuk mengkonversi geometri. Format ini siap digunakan langsung oleh Leaflet dan MapLibre di frontend.

- **/api/fasilitas/nearby**

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/fasilitas/nearby?lat=-5.3628&lon=105.3112&radius=1000' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/fasilitas/nearby?lat=-5.3628&lon=105.3112&radius=1000
```

Server response

Code | Details

200

Response body


```
[
  {
    "id": 7,
    "nama": "Institut Teknologi Sumatera",
    "jenis": "Universitas",
    "jarak_m": 95
  }
]
```


Download

Response headers

```
content-length: 82
content-type: application/json
date: Sat,04 Apr 2026 16:05:09 GMT
server: uvicorn
```

Endpoint query spasial untuk mencari fasilitas dalam radius tertentu dari koordinat yang diberikan. Menggunakan fungsi PostGIS ST_DWithin() yang memanfaatkan spatial index GIST sehingga lebih cepat dibanding ST_Distance. Hasil diurutkan berdasarkan jarak terdekat.

- **/api/fasilitas/{id} Ambil data Berdasarkan ID**

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/fasilitas/7' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/fasilitas/7
```

Server response

Code | Details

200

Response body


```
{
  "id": 7,
  "nama": "Institut Teknologi Sumatera",
  "jenis": "Universitas",
  "alamat": "Jl. Terusan Ryacudu, Way Huwi, Lampung Selatan",
  "longitude": 105.3107,
  "latitude": -5.3621
}
```


Download

Endpoint ini mengambil satu fasilitas berdasarkan ID. Menggunakan parameterized query WHERE id = $1 untuk mencegah SQL injection. Jika ID tidak ditemukan, mengembalikan error 404 Not Found

- **/api/fasilitas/{id} update**

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/api/fasilitas/4' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "nama": "Unila",
  "jenis": "Universitas",
  "alamat": "Jl. Prof. Sumantri Brojonegoro No.1",
  "longitude": 105.2412,
  "latitude": -5.3665
}'
```

Request URL

```
http://127.0.0.1:8000/api/fasilitas/4
```

Server response

Code	Details
200	Response body

```
{
  "id": 4,
  "nama": "Unila",
  "jenis": "Universitas",
  "alamat": "Jl. Prof. Sumantri Brojonegoro No.1",
  "longitude": 105.2412,
  "latitude": -5.3665
}
```

Download

Response headers

```
content-length: 132
content-type: application/json
date: Sat,04 Apr 2026 16:07:52 GMT
server: uvicorn
```

Endpoint PUT digunakan untuk memperbarui data fasilitas yang sudah ada. Menggunakan klausa RETURNING agar data yang telah diupdate langsung dikembalikan tanpa perlu query tambahan. Mengembalikan 404 jika ID tidak ditemukan

- **/api/fasilitas/{id} Delete**

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/api/fasilitas/4' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/api/fasilitas/4
```

Server response

Code	Details
204	Response headers

```
content-type: application/json
date: Sat,04 Apr 2026 16:09:38 GMT
server: uvicorn
```

Endpoint DELETE menghapus fasilitas berdasarkan ID. Sesuai konvensi REST, mengembalikan status 204 No Content (tanpa body response) jika penghapusan berhasil.

KESIMPULAN:

Secara keseluruhan, seluruh endpoint yang dibutuhkan telah berhasil diimplementasikan dan diuji dengan baik melalui Swagger UI, mencakup operasi CRUD lengkap serta fitur query spasial. Penggunaan *asyncpg* dengan connection pool memberikan performa yang optimal dalam pengelolaan koneksi database, sementara validasi data yang dilakukan oleh Pydantic memastikan keandalan dan konsistensi input. Selain itu, pemanfaatan fungsi PostGIS seperti

ST_DWithin dan *ST_AsGeoJSON* memungkinkan pemrosesan data spasial yang efisien serta menghasilkan output yang sesuai dengan standar GeoJSON internasional. Dengan demikian, sistem yang dibangun sudah memenuhi kebutuhan fungsional dan memiliki performa serta keakuratan yang baik dalam pengelolaan data spasial.